

Shahoshi

From Wokwi Simulation to Real Wearable Device

Complete Step-by-Step Build Guide

Asif Zaman

25-93713-1@student.aiub.com

Overview

This guide is the complete sequence for taking the project from a validated Wokwi simulation to a physically wearable device. It is organized into ten checkpointed phases — each phase must work correctly before moving to the next. Skipping ahead to enclosure assembly before the electronics are proven on a bench is the most common cause of failed first-attempt wearable builds, so the phases are deliberately ordered to prevent that.

Phase 0 — Tools & Parts Checklist

Have all of the following on hand before starting. Building without them causes most first-attempt failures.

Tools

- ☐ Soldering iron (temperature-controlled) + solder + flux
- ☐ Multimeter — continuity test and voltage check are non-negotiable
- ☐ Wire strippers / cutters
- ☐ Breadboard + jumper wires
- ☐ Small Phillips screwdriver set
- ☐ Hot glue gun or double-sided foam tape
- ☐ Helping-hands or PCB vice (optional, makes soldering easier)

Parts (from the finalized Bill of Materials)

- ☐ ESP32 DevKit V1
- ☐ MPU-6050 (GY-521)
- ☐ MAX30100 breakout
- ☐ LM393 sound sensor (analog output version, confirmed A0 pin)
- ☐ NEO-6M GPS module (confirmed antenna attached)
- ☐ Passive buzzer
- ☐ TP4056 charging module + 500mAh LiPo battery
- ☐ Jumper wires, perfboard/stripboard, header pins

Software already prepared

- ☐ phase1_sensor_diagnostic.ino
- ☐ sheshield_final_firmware.ino + feature_extractor.h
- ☐ Training pipeline: prepare_dataset.py, train_model.py, convert_to_tflite.py, convert_to_header.py

Phase 1 — Bench Test Every Sensor Individually

This repeats the Wokwi validation, but now with real hardware. Isolate failures here before wiring everything together.

1. Power up the bare ESP32 alone. Confirm normal boot messages on Serial Monitor at 115200 baud. If nothing shows, try a different USB cable — many are charge-only with no data lines.
2. Wire only the MPU-6050 (VCC→3.3V, GND→GND, SDA→GPIO21, SCL→GPIO22). Flash the Phase 1 diagnostic sketch. Confirm AZ reads ~1.0g at rest.
3. Add the MAX30100 to the same I2C bus. Place a finger on it and confirm non-zero BPM within a few seconds. Constant 0.0 readings indicate the known pull-up resistor issue — desolder R1/R2/R3 on the module.
4. Add the sound sensor (A0→GPIO34). Clap nearby and confirm the raw analog reading spikes.
5. Add the buzzer. Confirm it audibly pulses on boot.
6. Add the GPS module last (TX→GPIO16). Test outdoors or near a window and allow 30-60 seconds for a fix.

Checkpoint: do not proceed to Phase 2 until every sensor here reads sane values. A sensor that only "sort of" works now will produce unusable training data later.

Phase 2 — Full System on Breadboard with Final Firmware

1. Wire all five peripherals simultaneously per the pin map in sheshield_final_firmware.ino's header comment.
2. Fill in the firmware config block: WiFi credentials, Brevo API key, template ID, emergency number. Use placeholder/dummy model_data.h and scaler_data.h for now — this phase tests wiring and alerting, not model accuracy.
3. Flash and confirm in Serial Monitor: MPU6050 and MAX30100 both initialize, GPS eventually reports a fix, and WiFi connects.
4. Manually force a test alert (temporarily lower MOVE_THRESH, or add a debug trigger) to confirm the WhatsApp message arrives with a working Google Maps link.
5. Revert thresholds to normal once confirmed.

Checkpoint: this is the breadboard-complete milestone. Everything from here forward is about durability and physical form, not correctness.

Phase 3 — Real Data Collection

This step cannot be simulated — it must be performed with the real device on real volunteers.

1. Flash the data-logging firmware (Section 5.5 of the original guide) in place of the final firmware, temporarily.
2. Run the labeled collection procedure — walking, standing, sitting, running, controlled struggle, controlled fall, panic movement — on real volunteers wearing the real device.

3. Save each session as CSV. Target 30-50 volunteers; a model trained on 3-4 people will not generalize.
-

Phase 4 — Train the Model on a PC

1. Run `prepare_dataset.py` on the collected CSVs.
2. Run `train_model.py`. Check the confusion matrix and recall — recall (catching real danger) matters more than raw accuracy.
3. Run `convert_to_tflite.py`. Confirm the quantized model size is well under 100KB.
4. Run `convert_to_header.py` and `convert_scaler_to_header.py` to produce the real `model_data.h` and `scaler_data.h`.
5. Drop these into the firmware project, replacing the Phase 2 placeholder stubs.

Phase 5 — Re-Flash with the Real Model and Re-Test

1. Flash the updated firmware containing the real trained model.
2. Re-run the Phase 2 test scenarios, plus genuine walking, light exercise, and a supervised struggle simulation, to confirm the two-of-three consensus behaves correctly.
3. Tune MOVE_THRESH, HR_THRESH, SOUND_THRESH, and DANGER_CONFIRM_MS based on observed behavior. Multiple iterations here are expected and normal.

Checkpoint: do not move to permanent assembly until this phase is stable. Firmware threshold changes remain a simple re-flash even after Phase 6, but wiring changes become much harder once soldered.

Phase 6 — Harden the Circuit off the Breadboard

1. Get perfboard/stripboard sized to comfortably fit all components.
 2. Sketch the layout on paper first — plan where each component sits and route wires mentally before soldering.
 3. Solder the ESP32 (or a smaller WROOM/mini module if miniaturizing at this stage), then MPU-6050, MAX30100, sound sensor, GPS, and buzzer, using short rigid leads.
 4. Check continuity with a multimeter after each component — confirm no solder bridges, especially on the ESP32's tightly-spaced header.
 5. Power on and re-run the Phase 1 diagnostic checks on this soldered version to catch cold joints or bridged pins.
-

Phase 7 — Custom PCB (Recommended for Regular Wear)

1. Redraw the proven schematic in KiCad (free).
2. Lay out a PCB shaped for the wrist, positioning the MAX30100 at one edge for skin contact.
3. Order from JLCPCB or PCBWay — typically \$5-15 for 5 boards, about a week turnaround.
4. Solder components on arrival and re-run Phase 1 diagnostics a third time.

This step is optional for a demo/test prototype — stripboard from Phase 6 is wearable enough for short sessions. A custom PCB is what makes it durable enough for daily wear.

Phase 8 — Battery and Charging Integration

1. Confirm the TP4056 module charges the LiPo correctly on the bench — check the charge LED and verify rising battery voltage with a multimeter.
 2. Wire the LiPo's output through the TP4056 to the ESP32's 5V/VIN pin, not directly to 3.3V.
 3. Disconnect USB and confirm the device runs on battery alone. Measure runtime under normal operation and adjust sampling/GPS polling rates if battery life is too short.
-

Phase 9 — Enclosure and Physical Assembly

1. Measure the actual soldered board or PCB and adjust the enclosure model's parameters before printing — the initial version was sized generically.
2. Print base and lid (PETG or ABS recommended).
3. Test-fit the bare board before final assembly — confirm the MAX30100 window aligns with the sensor's actual position, reprint if off by even a few millimeters.
4. Secure the board and battery with foam tape or small screw posts — avoid hot glue directly on components that may need future access.
5. Thread a standard 20mm strap through the lugs.
6. Close the case and test fit on an actual wrist.

Phase 10 — Final Wear Test and Calibration

1. Wear the assembled device for an extended session, observing buzzer/alert behavior.
 2. Go through normal daily activities (walking, sitting, light exercise) to confirm no false alarms.
 3. Run one more supervised, consented struggle simulation with the fully assembled device to confirm the full alert path — buzzer, GPS fix, WhatsApp message with location — works in final physical form.
 4. Note any threshold drift from skin contact quality or wrist position, and make final firmware adjustments.
-

Summary Checklist

- ☐ Phase 1 — every sensor verified individually on bench
- ☐ Phase 2 — full system works on breadboard with final firmware
- ☐ Phase 3 — real data collected from volunteers
- ☐ Phase 4 — model trained and converted
- ☐ Phase 5 — real model tested and thresholds tuned on breadboard
- ☐ Phase 6 — circuit hardened onto stripboard, re-verified
- ☐ Phase 7 — (optional) custom PCB ordered and assembled
- ☐ Phase 8 — battery/charging confirmed working
- ☐ Phase 9 — enclosure printed, fitted, assembled
- ☐ Phase 10 — final wear test and calibration complete